WebdriverIO MCP

Guía resumida: MCP + WebdriverIO + Appium + OpenCode Assist

Automatización de navegador y apps móviles desde opencode con `@wdio/mcp`, más la suite E2E del proyecto (**31 tests**) y los asistentes de `.opencode/`. Comandos para **macOS y Windows**.

> Estado actual (macOS): Node 20.18.1 · Appium 2.19.0 · uiautomator2 3.5.2 · `emulator-5554` · suite **31/31 en verde**.

---

1. MCP en 30 segundos

`@wdio/mcp` expone tools para Chrome, apps nativas iOS/Android (vía Appium) y WebViews. Modelo de **sesión única**; las sesiones con `noReset: true` se auto-desvinculan.

Config global de opencode:

- **macOS/Linux:** `~/.config/opencode/opencode.jsonc`
- **Windows:** `%USERPROFILE%\.config\opencode\opencode.jsonc`

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "wdio-mcp": { "type": "local", "command": ["npx", "-y", "@wdio/mcp"], "enabled": true }
  }
}
```

```
opencode mcp list
```

Variables de Appium: `APPIUM_URL` (127.0.0.1), `APPIUM_URL_PORT` (4723 oficial; **4724 en este proyecto**), `APPIUM_PATH` (`/`).

2. Herramientas clave

Área	Tools
Sesión	`start_session`, `attach_session`, `close_session`
Navegador	`launch_chrome`, `navigate`, `click_element`, `set_value`, `get_elements`, `get_accessibility_tree`, `get_screenshot`, `scroll`, `get_tabs` / `switch_tab`, cookies, `execute_script`
Móvil	`tap_element`, `swipe`, `drag_and_drop`, `get_app_state`, `get_contexts` / `switch_context`, `rotate_device`, `set_geolocation`, `hide_keyboard`, `execute_script` (`mobile: ...`)
Extras	`upload_app` / `list_apps`, `mock` / `manage_mock`, `query_docs`

> Nombres actuales: `start_session`, `get_elements`, `get_accessibility_tree`, `get_screenshot` y `launch_chrome` (antes `start_app_session`, `get_visible_elements`, `get_accessibility`, `take_screenshot`, `start_browser`).

3. Selectores

Preferir **Accessibility ID** (`~id`) en móvil. En web: CSS, XPath o `data-testid`. En Android también sirven UiAutomator/XPath e iOS predicates.

Pantalla	Selectores
Login	`~Login-screen`, `~input-email`, `~input-password`, `~button-LOGIN`
Forms	`~Forms-screen`, `~text-input`, `~switch`, `~Dropdown`
Swipe	`~Swipe-screen`, `~card`
Drag	`~Drag-drop-screen`, `~drag-l1` … `~drag-r3`, `~drop-l1` … `~drop-r3`, `~renew`, `~button-Retry`
Home / barra	`~Home-screen`, `~Home`, `~Webview`, `~Login`, `~Forms`, `~Swipe`, `~Drag`, `~Menu`

> No existe `~Carousel`; el carrusel se localiza con `~Swipe-screen` + `~card`.

4. Levantar el stack (macOS y Windows)

Requisitos: Node 20+, JDK 17, Android Studio (SDK + Platform-Tools + Emulator), un AVD, Appium 2 + `uiautomator2` y `DriverIO.apk` en la raíz.

```
# macOS
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator:$PATH"
```

```
# Windows PowerShell
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\Sdk"
$env:Path += ";$env:ANDROID_HOME\platform-tools;$env:ANDROID_HOME\emulator"
```

```
npm install
npx appium driver install uiautomator2
npm run emulator                # arranca el AVD y espera el boot
```

Emulador manual:

```
# macOS
emulator -avd Medium_Phone_API_36.1
```

```
# Windows
& "$env:LOCALAPPDATA\Android\Sdk\emulator\emulator.exe" -avd Medium_Phone_API_36.1
```

Appium para MCP (la suite lo levanta sola):

```
npx appium --port 4724
```

Verificación: `adb devices` → `emulator-5554 device`; `appium driver list --installed` → `uiautomator2`.

5. Ejecución de la suite (macOS y Windows)

Comando	Descripción
`npm run test:all`	Suite completa (31 tests)
`npm run test:login` · `test:forms` · `test:swipe` · `test:drag` · `test:home`	Por feature
`npm run test:smoke`	Validaciones visuales
`npx wdio run ./wdio.conf.ts --spec ./features/specs/login.spec.ts`	Un spec
`npx wdio run ./wdio.conf.ts --mochaOpts.grep "CP-019"`	Filtro por caso

```
# macOS / Linux
npm run test:all
ALLURE_REPORT=1 npm run test:all
npx allure serve allure-results
```

```
# Windows PowerShell
npm run test:all
$env:ALLURE_REPORT = "1"; npm run test:all; Remove-Item Env:ALLURE_REPORT
npx allure serve allure-results
```

Variable	Default	macOS/Linux	Windows PowerShell
`ANDROID_AVD`	primer AVD	`ANDROID_AVD=Medium_Phone_API_36.1 npm run test:all`	`$env:ANDROID_AVD="Medium_Phone_API_36.1"; npm run test:all`
`EMULATOR_BOOT_TIMEOUT`	`240000`	`EMULATOR_BOOT_TIMEOUT=360000 npm run test:all`	`$env:EMULATOR_BOOT_TIMEOUT="360000"; npm run test:all`
`ANDROID_DEVICE_NAME`	serial detectado	`ANDROID_DEVICE_NAME=emulator-5554 npm run test:all`	`$env:ANDROID_DEVICE_NAME="emulator-5554"; npm run test:all`
`ALLURE_REPORT`	desactivado	`ALLURE_REPORT=1 npm run test:all`	`$env:ALLURE_REPORT="1"; npm run test:all`

> En Windows CMD reemplaza `$env:VAR="valor";` por `set VAR=valor &&`. `npm run test:*` arranca emulador y Appium automáticamente.

6. OpenCode Assist (`.opencode/`)

Configuración versionada para asistir el desarrollo de la suite desde opencode:

Componente	Qué implementa
`plugins/harness.ts`	Plugin QA: tool **`qa_run_suite`** (suites `login/forms/swipe/drag/home/smoke/all`, `grep` Mocha y `allure`); inyecta `APPIUM_URL_PORT` y `APPIUM_PORT=4724`; registra capturas MCP como evidencia
`agent/appium-debugger.md`	Diagnóstico de emulador, Appium, driver, APK y logs
`agent/page-object-generator.md`	Genera Page Objects con accessibility ids, métodos async, `isOnScreen`, `validateElements` y singleton
`agent/test-writer.md`	Escribe specs Mocha/BDD con IDs `[CP-NNN]`, español y uso exclusivo del POM
`agent/test-reviewer.md`	Revisa mantenibilidad, flakiness, aserciones, cobertura y duplicación
`command/appium-check.md`	`/appium-check`: verifica adb, Appium, driver y APK; opcionalmente abre sesión MCP
`command/new-spec.md`	`/new-spec <feature>`: crea spec + page object + script npm
`command/run-tests.md`	`/run-tests <suite> [grep]`: ejecuta la suite o un filtro; con evidencias Allure
`skills/appium-setup`	Checklist del stack, variables, capabilities y troubleshooting
`skills/page-object-conventions`	Reglas de Page Objects y selectores conocidos
`skills/istqb-test-design`	Técnicas ISTQB: partición de equivalencia, valores límite, tablas de decisión, estados, pairwise
`skills/allure-reporting`	Generación, servicio e interpretación de reportes Allure

```
/run-tests all
/run-tests drag CP-019
/new-spec search
/appium-check
```

> El tool `qa_run_suite` ejecuta en el worktree: `qa_run_suite({ suite: "drag", grep: "CP-019", allure: true })`. Se versionan agents, commands, skills y el plugin; `.opencode/node_modules` y sus manifiestos quedan ignorados.

7. Troubleshooting

Síntoma	Solución
Emulador colgado / `invalid session id`	`adb emu kill` + `npm run emulator`
APK no encontrado	Copiar `DriverIO.apk` a la raíz
SDK no detectado en Windows	Definir `ANDROID_HOME` / `ANDROID_SDK_ROOT`
`~Carousel` no encontrado	Usar `~Swipe-screen` y `~card`
Alert de login bloquea la bottom bar	Pulsar `OK`
Teclado tapa la bottom bar	`hide_keyboard` / `driver.hideKeyboard()`
Modal “Congratulations” bloquea `Renew`	Pulsar `~button-Retry`
Appium no arranca / puerto ocupado	Liberar 4724 o usar `npx appium --port 4724`

8. Recursos

- MCP: https://webdriver.io/docs/mcp · https://github.com/webdriverio/mcp
- Tools y selectores: https://webdriver.io/docs/mcp/tools · https://webdriver.io/docs/mcp/selectors
- Especificación MCP: https://modelcontextprotocol.io

> *Resumen de la documentación oficial de WebdriverIO MCP (© OpenJS Foundation y colaboradores, 2026), adaptado al estado actual del proyecto.*